LIBRARY MANAGEMENT SYSTEM · UML 2.5

图书馆管理系统
UML 建模图集

十种视图 · 同一借阅业务场景贯通 · 需求 / 结构 / 行为三视角

建模对象	参与者	记法标准	图幅数量
高校图书馆管理系统	读者 · 图书管理员 · 系统管理员	UML 2.5	10 张

纯矢量绘制 · 可无损缩放 · 适用于课程作业、需求规格说明书与答辩材料

图目录

01	**用例图**	*Use Case Diagram*	参与者与系统功能边界
02	**时序图**	*Sequence Diagram*	借书场景的 13 条消息交互
03	**类图**	*Class Diagram*	八个核心类与关联、多重度
04	**对象图**	*Object Diagram*	某时刻的运行时快照
05	**活动图**	*Activity Diagram*	借书流程、判定与并发分叉
06	**Legacy 传统记法**	*Collaboration Diagram*	UML 1.x 协作图与新旧差异
07	**组件图**	*Component Diagram*	分层架构与依赖关系
08	**部署图**	*Deployment Diagram*	物理节点、制品与通信协议
09	**状态图**	*State Machine Diagram*	馆藏副本的全生命周期
10	**时间图**	*Timing Diagram*	状态随时间的量化约束

01 用例图（Use Case Diagram）

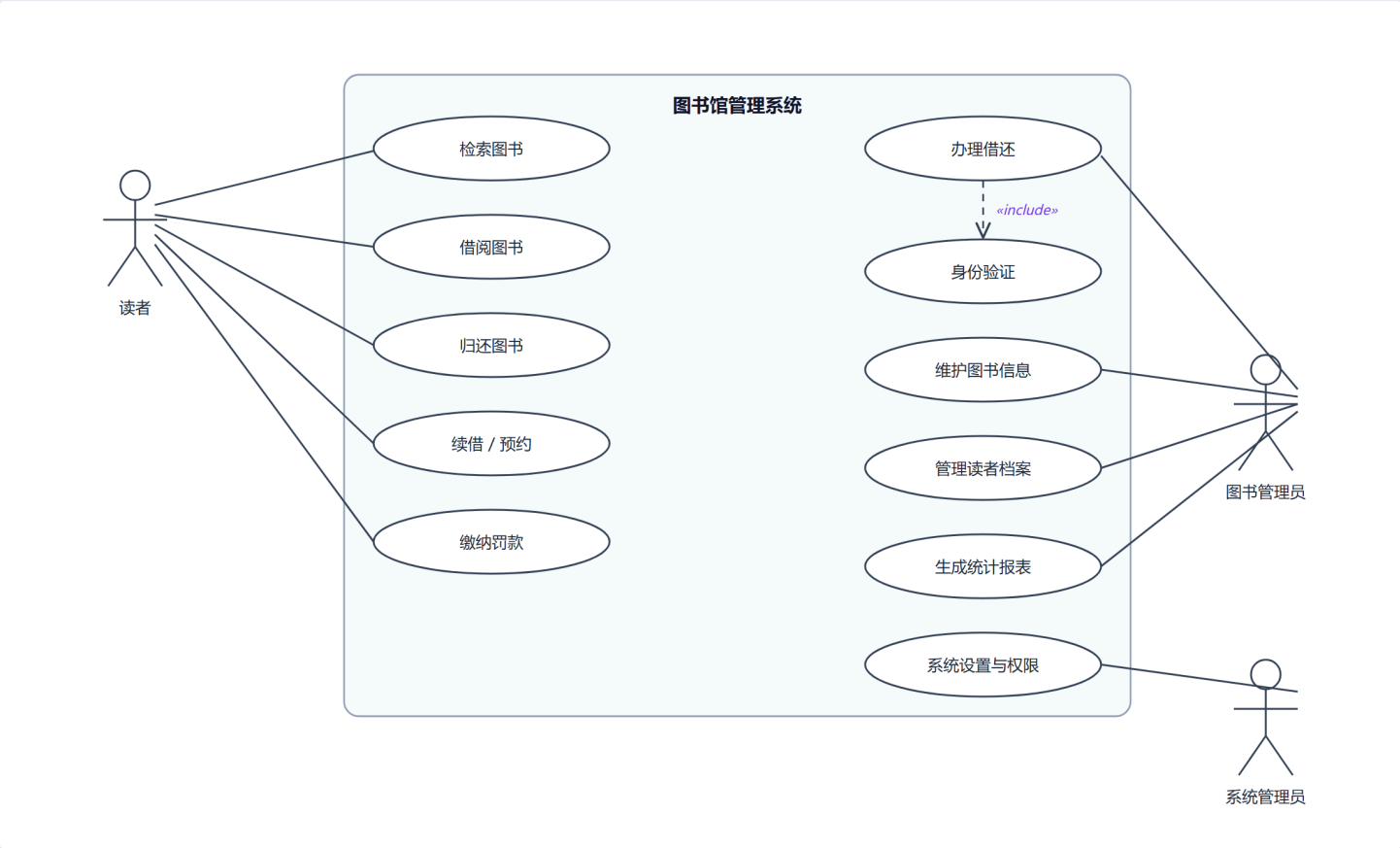


图 01 · 用例图

描述： 系统边界内的功能需求与外部参与者的交互。

- **读者** 可执行：检索图书、借阅图书、归还图书、续借/预约、缴纳罚款；
- **图书管理员** 可执行：办理借还、维护图书信息、管理读者档案、生成统计报表；
- **系统管理员** 可执行：系统设置与权限管理。

要点：

- 「办理借还」通过 **«include»** 强制包含「身份验证」——每次借还都必须校验读者身份；若写成 «extend» 则表示可选扩展行为（如「代借」扩展「办理借还」）。
- UML 2.x 中参与者与用例之间只画**关联线**，不画箭头。

---

02 时序图（Sequence Diagram）

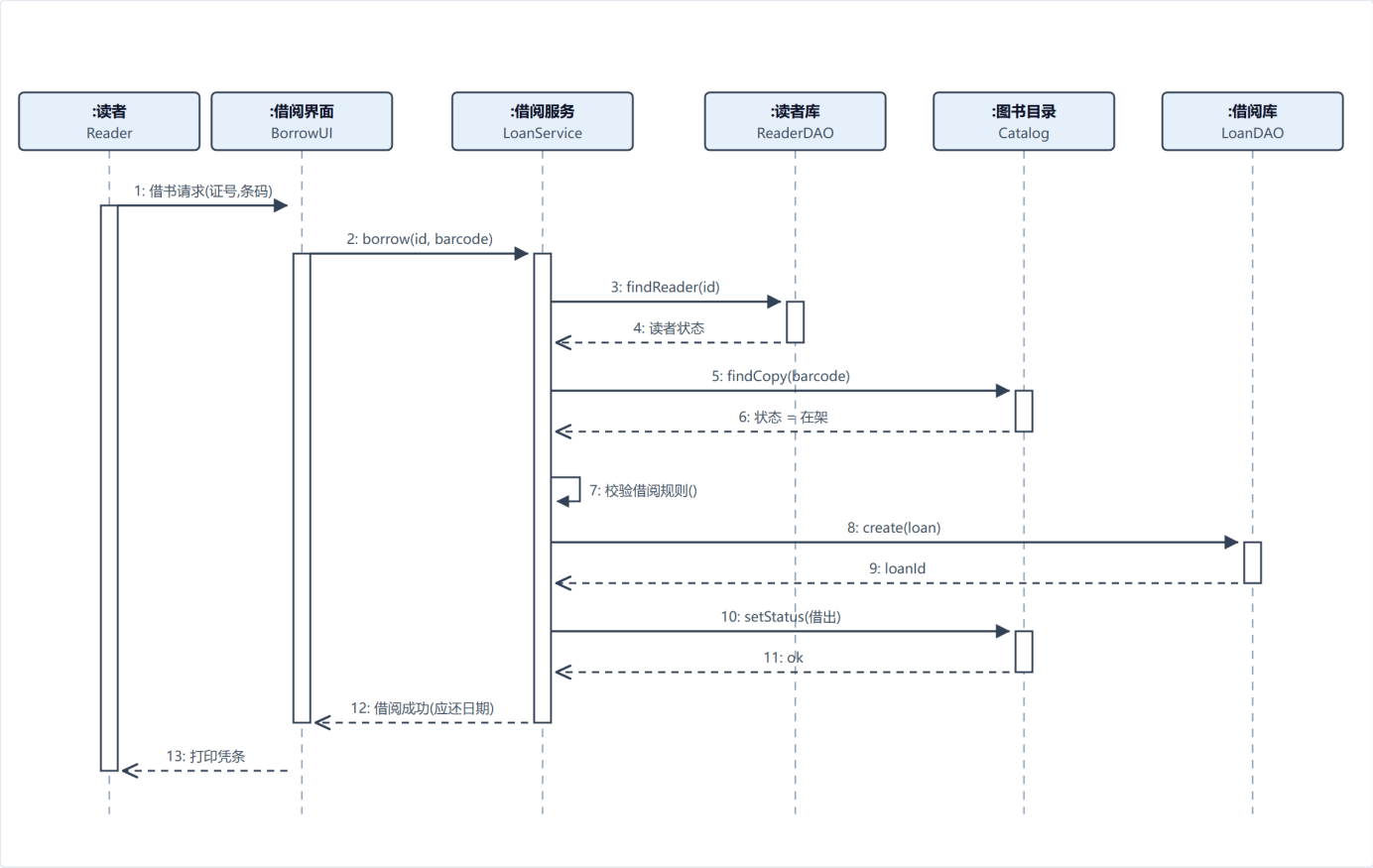


图 02 · 时序图

描述： 「读者借书」场景的对象交互，共 13 条消息。

阶段	消息	说明
请求	1–2	读者 → 借阅界面 → 借阅服务 `borrow(id, barcode)`
校验	3–7	查读者状态 → 查副本状态 → 自调用校验借阅规则

阶段	消息	说明
落库	8–11	创建借阅记录 → 更新副本状态为「借出」
反馈	12–13	返回应还日期 → 打印凭条

要点： 消息编号、同步 / 返回两种箭头、激活期（执行控制焦点）是时序图的四大得分要素。读者必须先通过「读者状态」校验（消息 3–4）才能进入副本查询。

---

03 类图（Class Diagram）

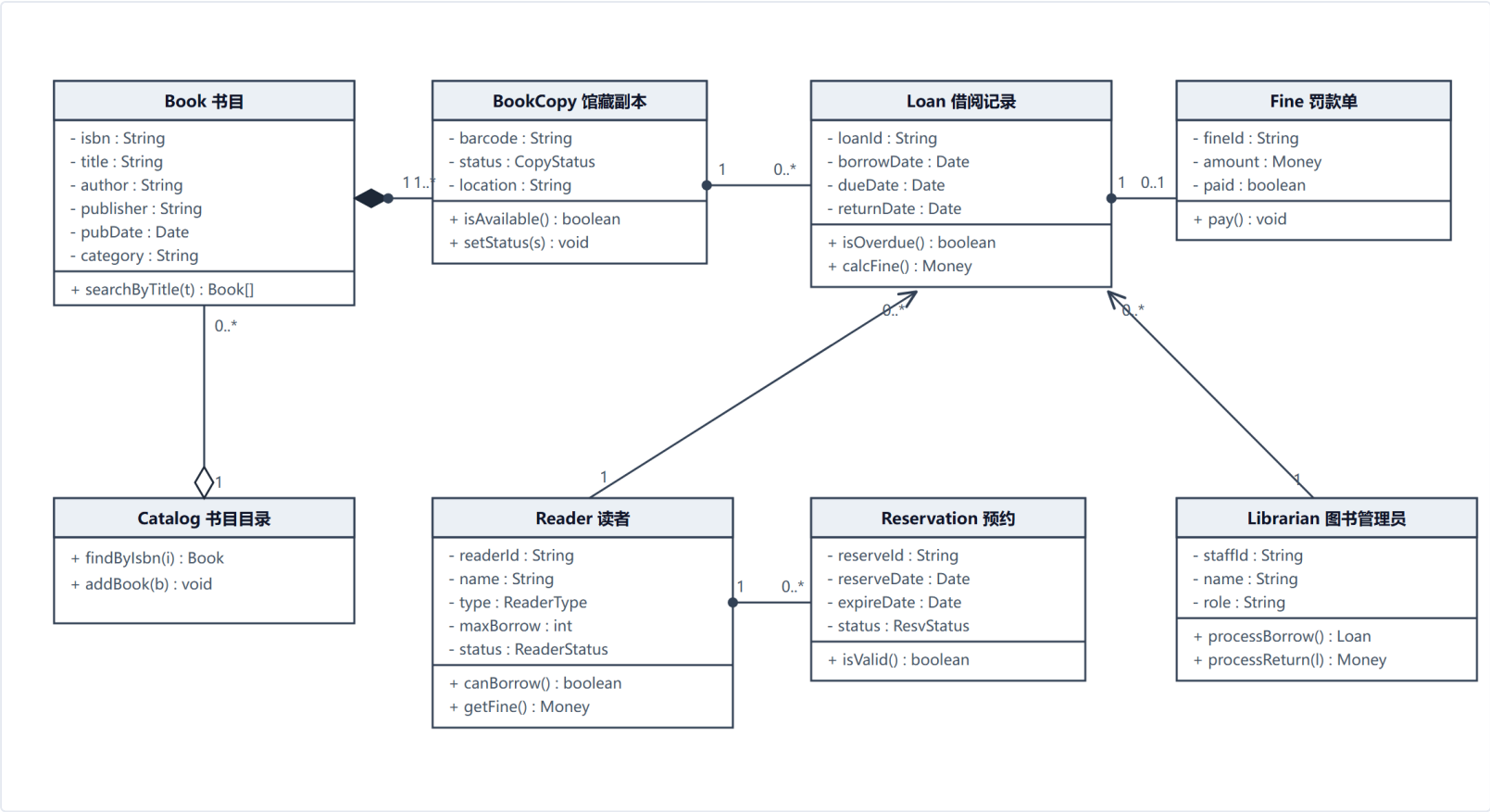


图 03 · 类图

描述： 系统静态结构，8 个核心类及其属性、操作与关联。

类	职责	关键属性
`Book` 书目	描述一种书的元数据	isbn、title、author、category
`BookCopy` 馆藏副本	描述物理上的一册	barcode、status、location

类	职责	关键属性
`Reader` 读者	借阅主体	readerId、type、maxBorrow
`Loan` 借阅记录	一次借还的流水	borrowDate、dueDate、returnDate
`Reservation` 预约	到书通知	reserveDate、expireDate
`Fine` 罚款单	逾期经济责任	amount、paid
`Librarian` 图书管理员	业务操作者	staffId、role
`Catalog` 书目目录	书目的组织容器	—

要点：

- ◆ **实心菱形** 是组合：书目被删除，其馆藏副本随之消亡；◇ **空心菱形** 是聚合：目录撤销不影响书目本身。
- 避免把「借阅记录」设计成「读者」的内部属性——它是独立实体，否则无法保留历史流水。

---

04 对象图（Object Diagram）

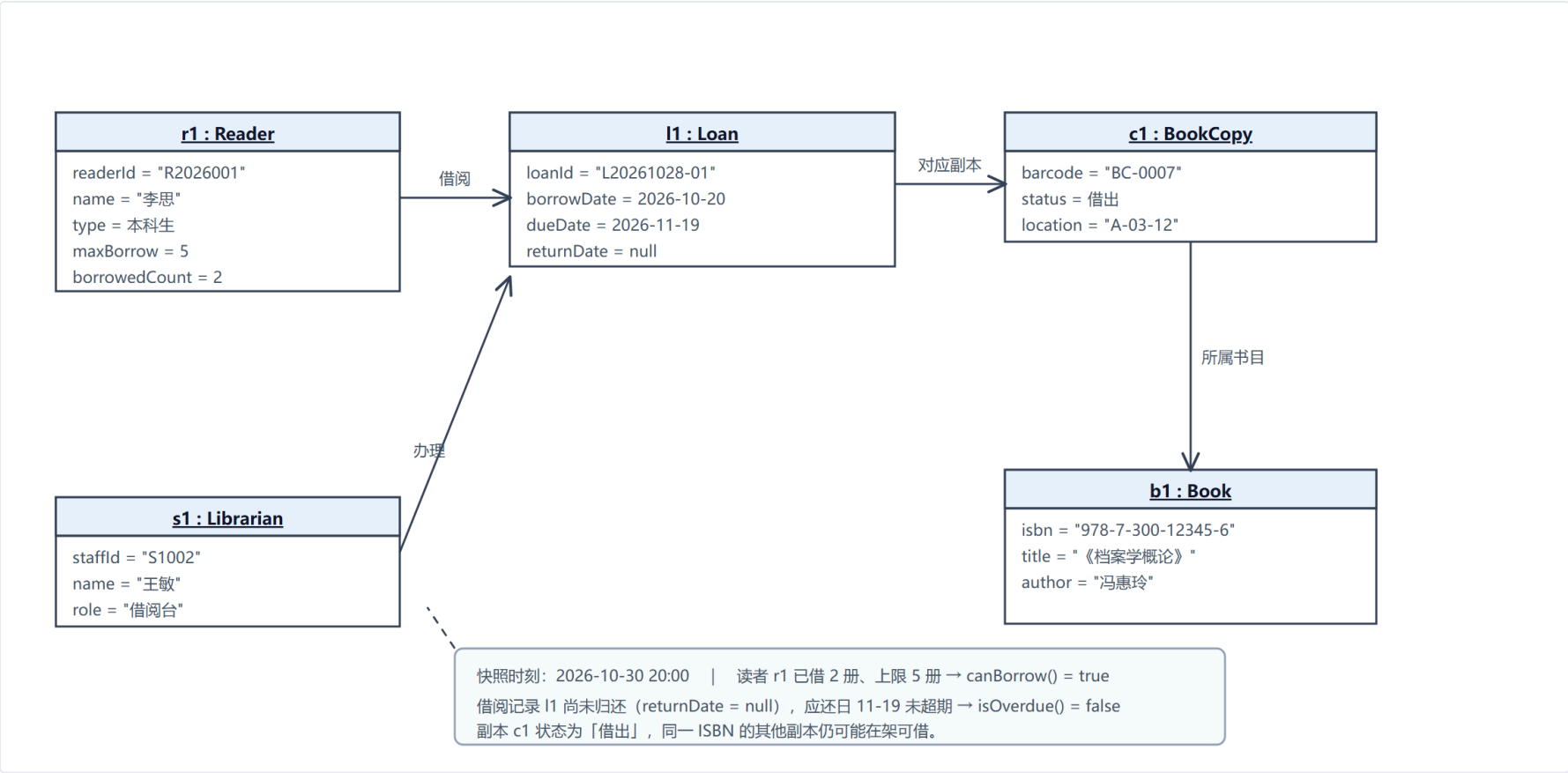


图 04 · 对象图

描述： 类图在 **2026-10-30 20:00** 时刻的运行时快照。

- 读者 `r1`（李思，本科生）已借 2 册、上限 5 册 → `canBorrow() = true`
- 借阅记录 `l1` 于 10-20 借出，应还日 11-19，尚未归还（`returnDate = null`）→ `isOverdue() = false`
- 副本 `c1`（BC-0007）状态为「借出」，位于 A-03-12 书架

要点： 对象图是「运行时快照」，**不带操作方法**（操作属于类的定义），只写属性值。它的价值在于验证类图的多重度是否成立：一个 Reader 对应 0..* 个 Loan，一个 BookCopy 对应 0..* 个 Loan。

---

05 活动图（Activity Diagram）

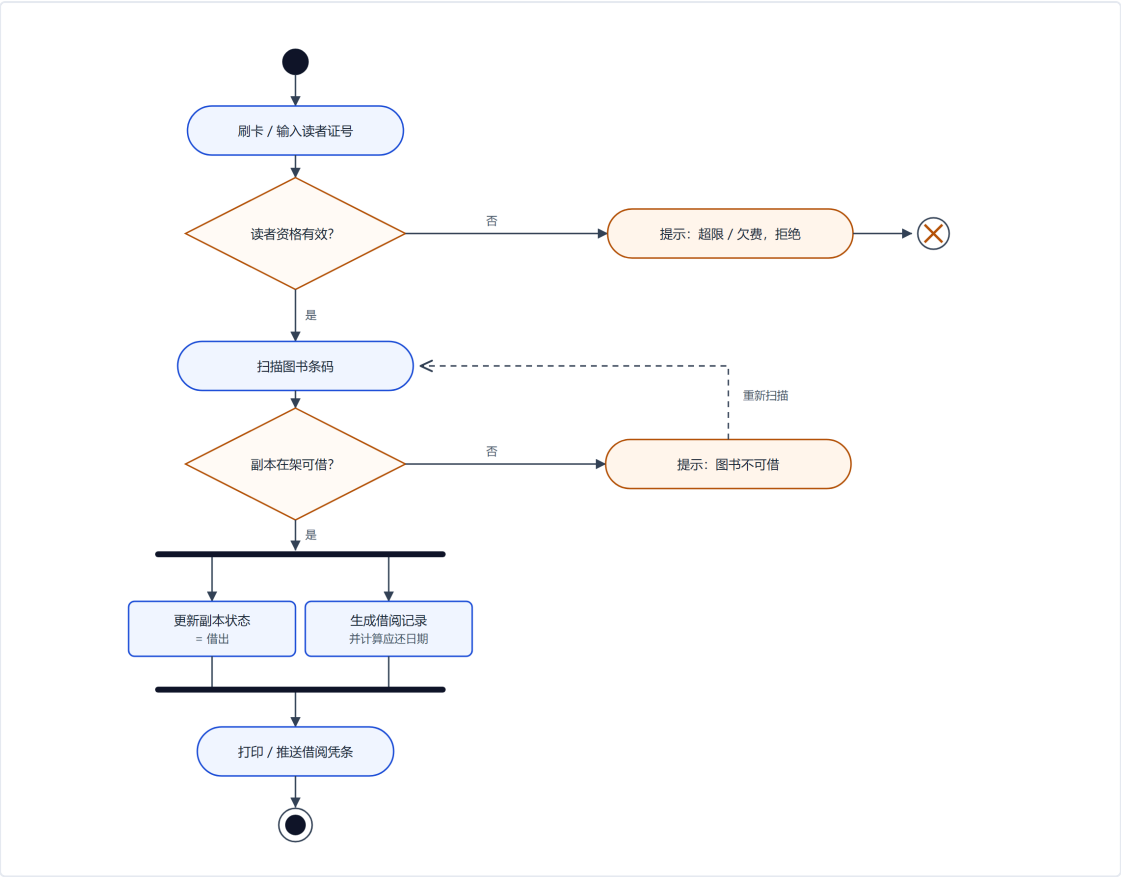


图 05 · 活动图

描述： 借书业务流程的控制流。

1. 刷卡 / 输入读者证号 → **判定**：读者资格有效?
2. 否 → 提示超限/欠费 → **终止**（中断）
3. 是 → 扫描图书条码 → **判定**：副本在架可借?
4. 否 → 提示图书不可借 → 虚线返回重新扫描

5. 是 → **并发分叉**：更新副本状态 || 生成借阅记录并计算应还日期 → **汇合**
6. 打印/推送借阅凭条 → 正常结束

要点： 菱形是判定（一个入流、多个带监护条件的出流）；粗黑线是并发分叉与汇合——「更新副本状态」和「生成借阅记录」必须同时成功或同时回滚，对应数据库事务。终止符（内含 ×）表示流程被中断，实心圆环表示正常结束。

---

06 Legacy 传统记法（协作图 / 通信图）

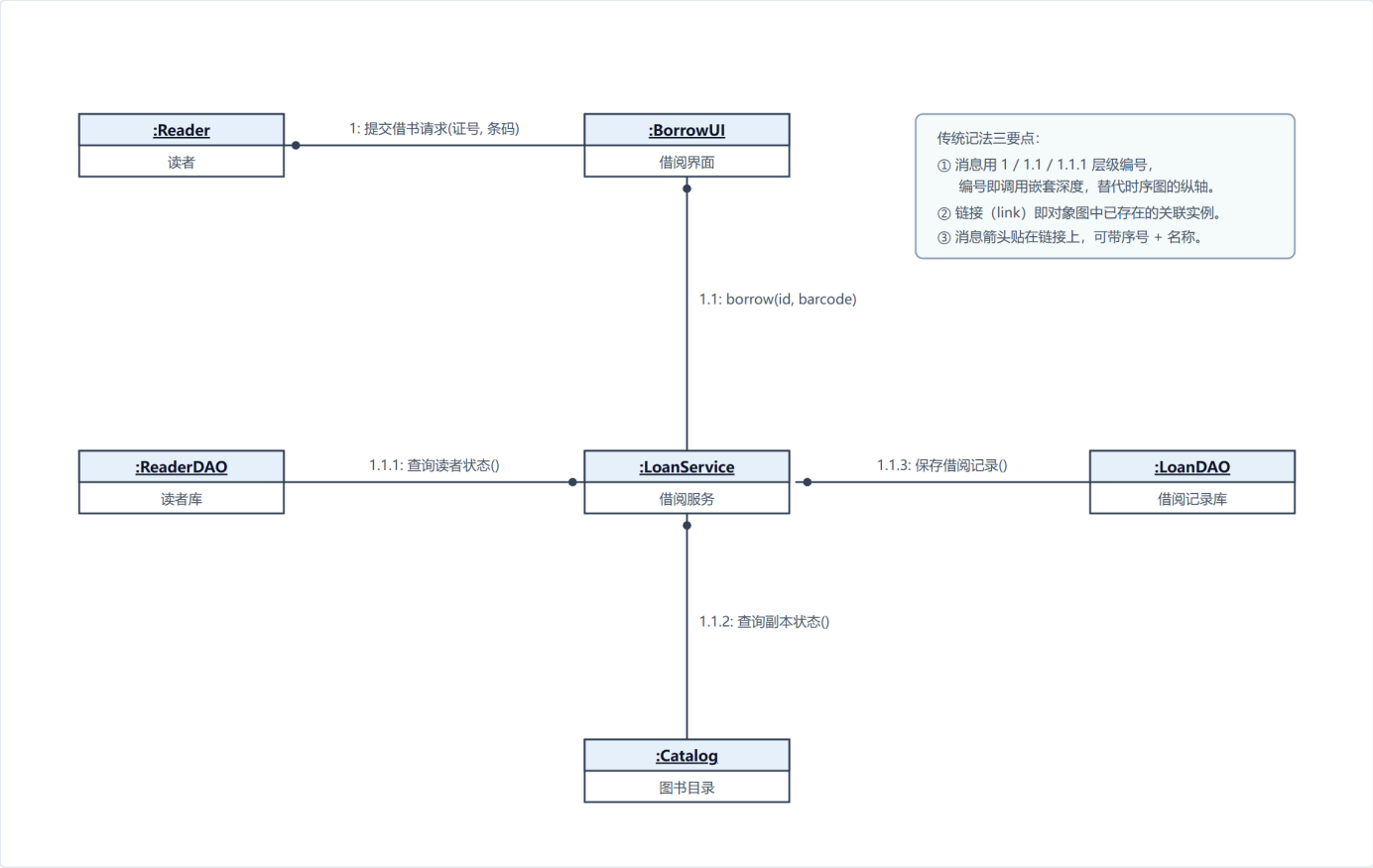


图 06 · 传统记法协作图

描述： UML 1.x 时代称「协作图（Collaboration Diagram）」，UML 2.x 更名「通信图（Communication Diagram）」。对象间用链接连线，消息以层级编号标注在链接上——它是时序图的等价表达。

消息链：`1: 提交借书请求` → `1.1: borrow()` → `1.1.1: 查询读者状态` / `1.1.2: 查询副本状态` / `1.1.3: 保存借阅记录`

传统记法三要点：

1. 消息用 `1 / 1.1 / 1.1.1` 层级编号，编号即调用嵌套深度，替代时序图的纵轴；

2. 链接（link）即对象图中已存在的关联实例；
3. 消息箭头贴在链接上，可带序号 + 名称。

其他 Legacy 差异：

UML 1.x	UML 2.x
协作图 Collaboration Diagram	通信图 Communication Diagram
«uses» 构造型	«include»
时序图不画激活条	必须画激活条
泳道内嵌写法	独立分区（Partition）元素

---

07 组件图（Component Diagram）

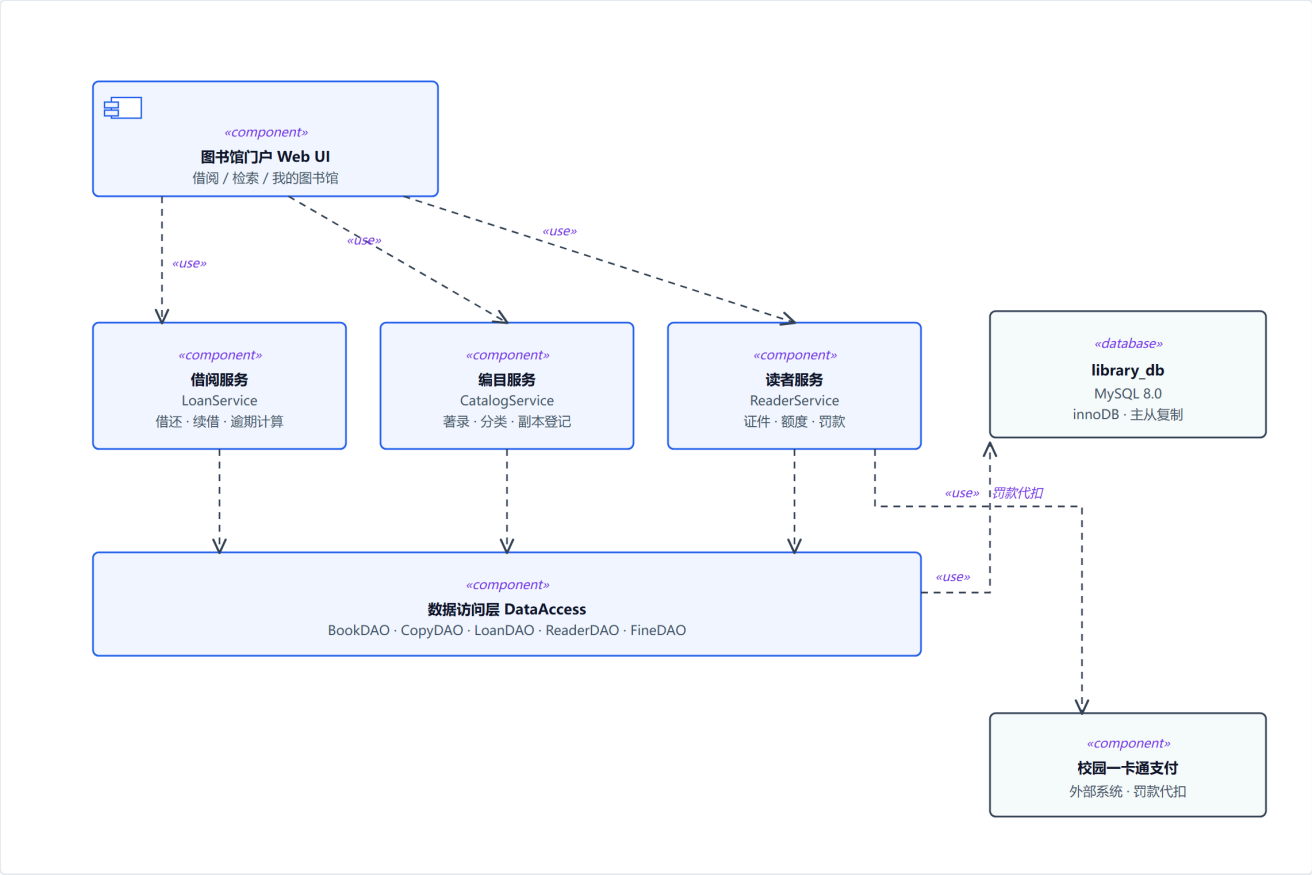


图 07 · 组件图

描述： 按分层架构组织可替换的软件单元。



要点： 组件是「可替换的物理实现单元」，接口是它的契约。虚线箭头 = 依赖（«use»），实线 + 棒棒糖（○）= 供接口，插座（◐）= 需接口。图中三层单向依赖，上层变更不会反向污染下层。

---

08 部署图（Deployment Diagram）

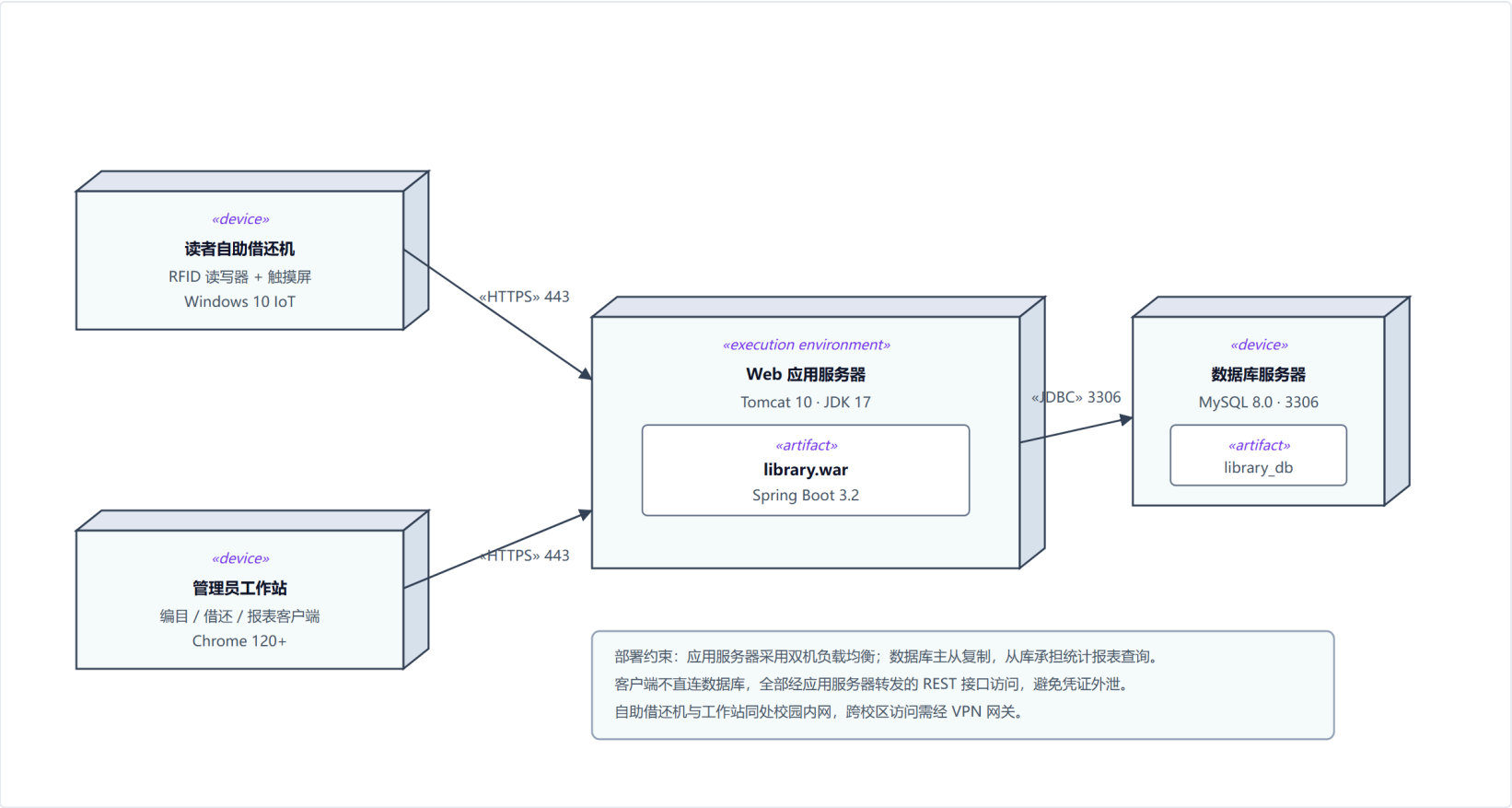


图 08 · 部署图

描述： 物理节点与制品（artifact）的分布，以及节点间的通信协议。

节点	类型	内容	通信
读者自助借还机	«device»	RFID 读写器 + 触摸屏，Windows 10 IoT	«HTTPS» 443 → 应用服务器
管理员工作站	«device»	编目/借还/报表客户端，Chrome 120+	«HTTPS» 443 → 应用服务器

节点	类型	内容	通信
Web 应用服务器	«execution environment»	Tomcat 10 · JDK 17，制品 `library.war`（Spring Boot 3.2）	«JDBC» 3306 → 数据库
数据库服务器	«device»	MySQL 8.0，制品 `library_db`	—

部署约束： 应用服务器双机负载均衡；数据库主从复制，从库承担统计报表查询；客户端不直连数据库，全部经应用服务器转发；跨校区访问经 VPN 网关。

要点： 部署图回答「程序装在哪台机器上」。节点用立体框，内部再嵌 «artifact» 制品；连线是通信路径，必须标协议与端口。三层部署（客户端 / 应用 / 数据库）是图书馆这类 B/S 系统的标准形态。

---

09 状态图（State Machine Diagram）

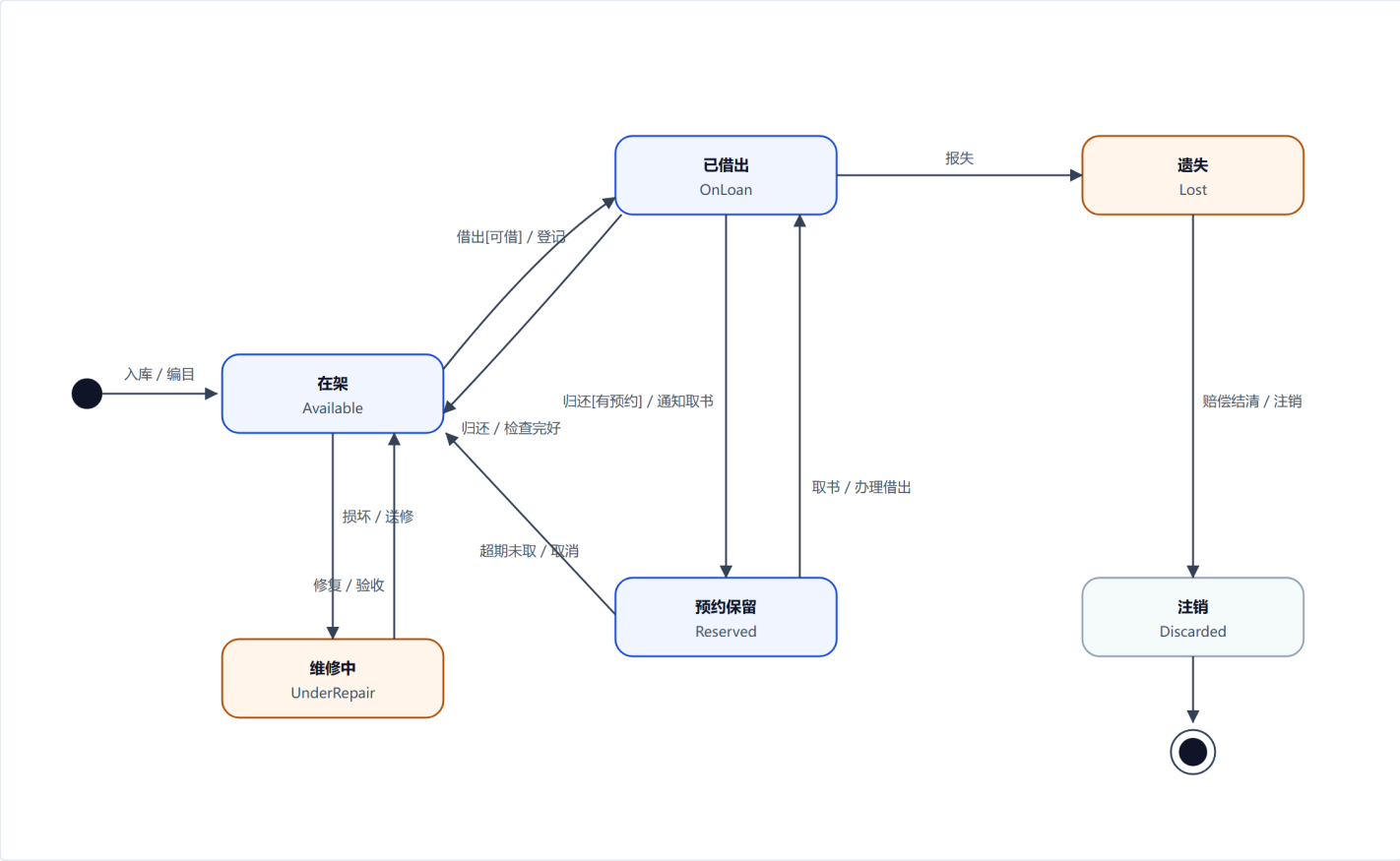


图 09 · 状态图

描述： 以「馆藏副本 BookCopy」为对象，刻画其全生命周期。

转换	事件 [守卫] / 动作
初始 → 在架	入库 / 编目
在架 → 已借出	借出 [可借] / 登记

转换	事件 [守卫] / 动作
已借出 → 在架	归还 / 检查完好
已借出 → 预约保留	归还 [有预约] / 通知取书
预约保留 → 已借出	取书 / 办理借出
预约保留 → 在架	超期未取 / 取消
在架 ⇄ 维修中	损坏 / 送修 ｜ 修复 / 验收
已借出 → 遗失	报失
遗失 → 注销 → 终态	赔偿结清 / 注销

要点： 状态图描述「一个对象」的状态变迁，与活动图（一条流程的控制流）不同。转换上的三段式 **事件[守卫条件]/动作** 是得分点：如「归还[有预约]/通知取书」——只有该副本已被预约时才进入「预约保留」，否则直接回「在架」。

---

10 时间图（Timing Diagram）

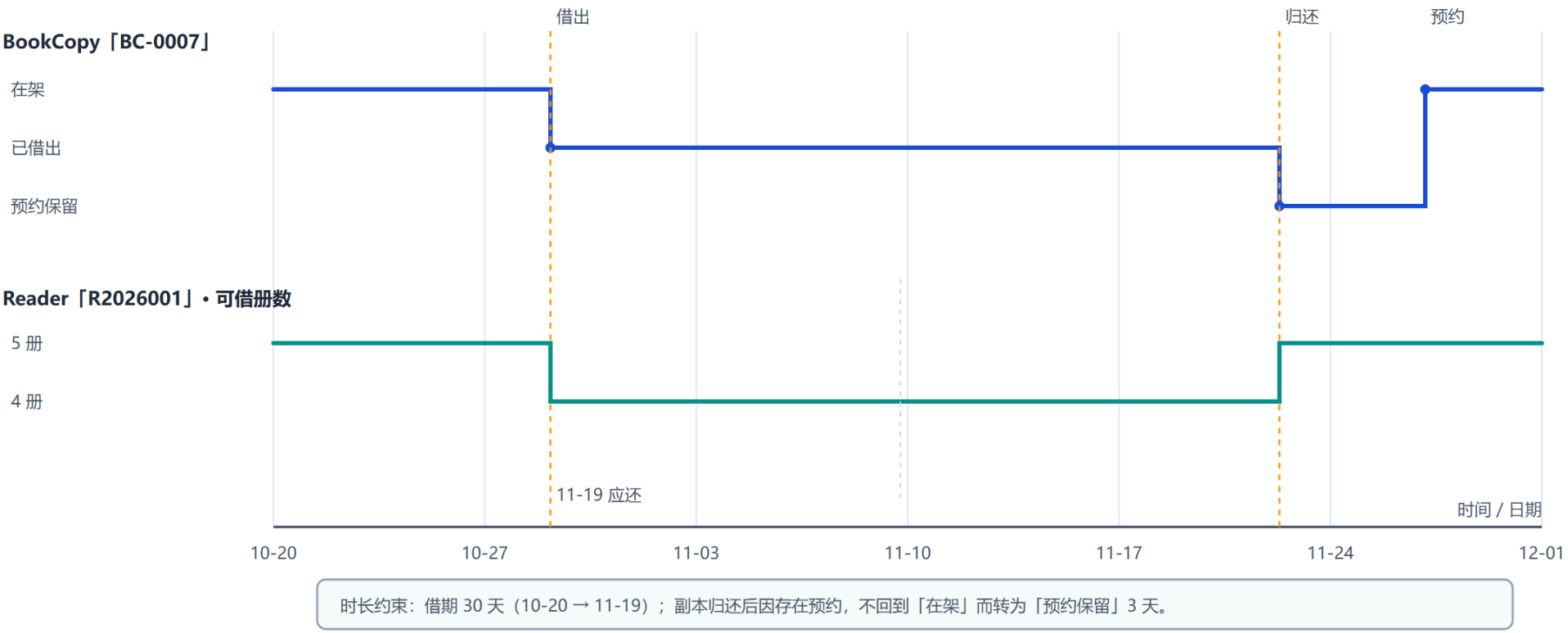


图 10 · 时间图

描述： 以时间为横轴，展示两个对象状态的阶梯式变化（10-20 → 12-01）。

- **BookCopy「BC-0007」：** 在架 →（10-27 借出）→ 已借出 →（11-24 归还）→ 预约保留 →（3 天后）→ 在架
- **Reader「R2026001」可借册数：** 5 册 →（借出时）4 册 →（归还时恢复）5 册

时长约束： 借期 30 天（10-20 → 11-19）；副本归还后因存在预约，不回到「在架」而转为「预约保留」3 天。

要点： 时间图的价值是「量化」——它把「借期 30 天」「预约保留 3 天」这类业务规则画成可测量的线段，比状态图更适合表达时限、超时、并发占用的约束。

附：十图之间的逻辑关系

```
需求视角    01 用例图 ──────────────── 谁要用系统、用系统做什么
              │
结构视角    03 类图 ← 04 对象图        静态结构 / 某时刻的快照
            07 组件图 ← 08 部署图      软件单元 / 装到哪台机器
              │
行为视角    02 时序图 ⇄ 06 协作图      按时间交互 ⇄ 按结构交互（等价）
            05 活动图                  一条流程的控制流
            09 状态图                  一个对象的状态变迁
            10 时间图                  状态随时间的量化约束
```

纯 SVG 手绘 · 可离线查看 / 打印 / 转 PDF ｜ 图片目录：`diagrams/`